

R Workshop

Clustering concepts and correlation

Presented By

Nick Ceglia
Katie Campbell

Date

May 17, 2026

Data

In a tibble or data.frame

- Rows are observations - the data points in our data.
- Columns are variables that describe the data including:
 - Features
 - Labels or targets
 - Identifiers
 - Derived features
- A set of features for a set of observations becomes our input to different “downstream” analysis.
- In the iris dataset.
 - Each row is a single flower.
 - There are 4 features.
 - There is one label with 3 classes.

row	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa

```
data(iris)
head(iris)

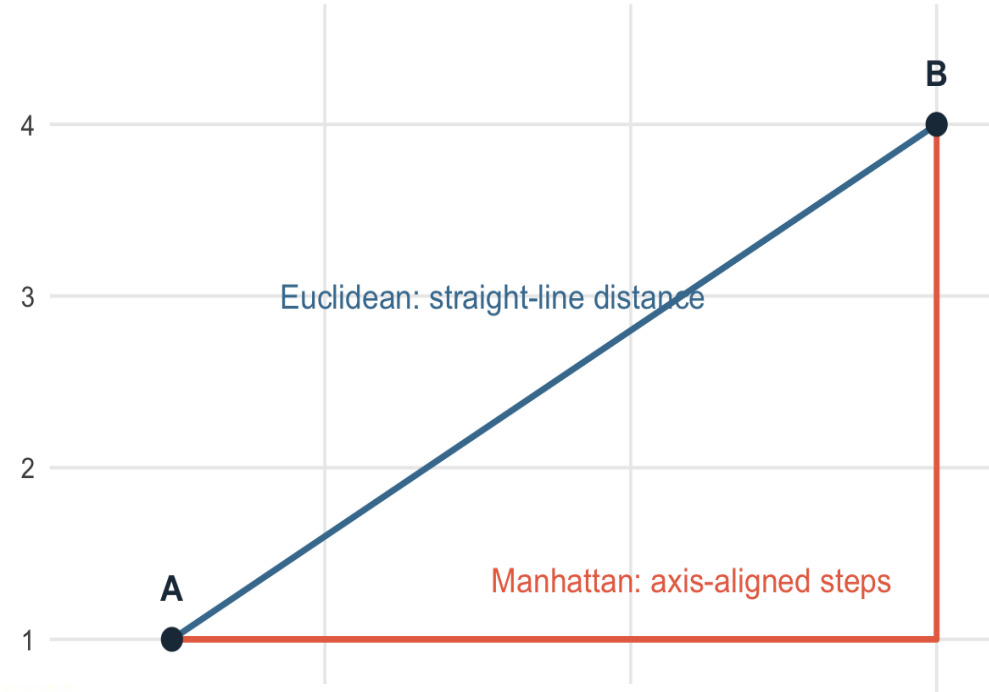
X <- iris[, 1:4]    # numeric features
labels <- iris$Species # metadata/known label
```

Distance or Similarity

How close or far are two points?

- How do we decide if two points are similar?
 - Euclidean distance.
 - Shortest path (L2)
- Other distance metrics
 - Manhattan (L1)
 - Robust to outliers
 - Less sensitive to difference
 - Cosine distance
 - High dimension and sparse
 - Normalization (Angle)
 - String distance
 - Levenshtein
 - How many character changes to make two strings equivalent

Distance metrics define what "similar" means



dist in R

```
# Load the iris dataset
data(iris)
iris_numeric <- iris[, 1:4] # Exclude the species column

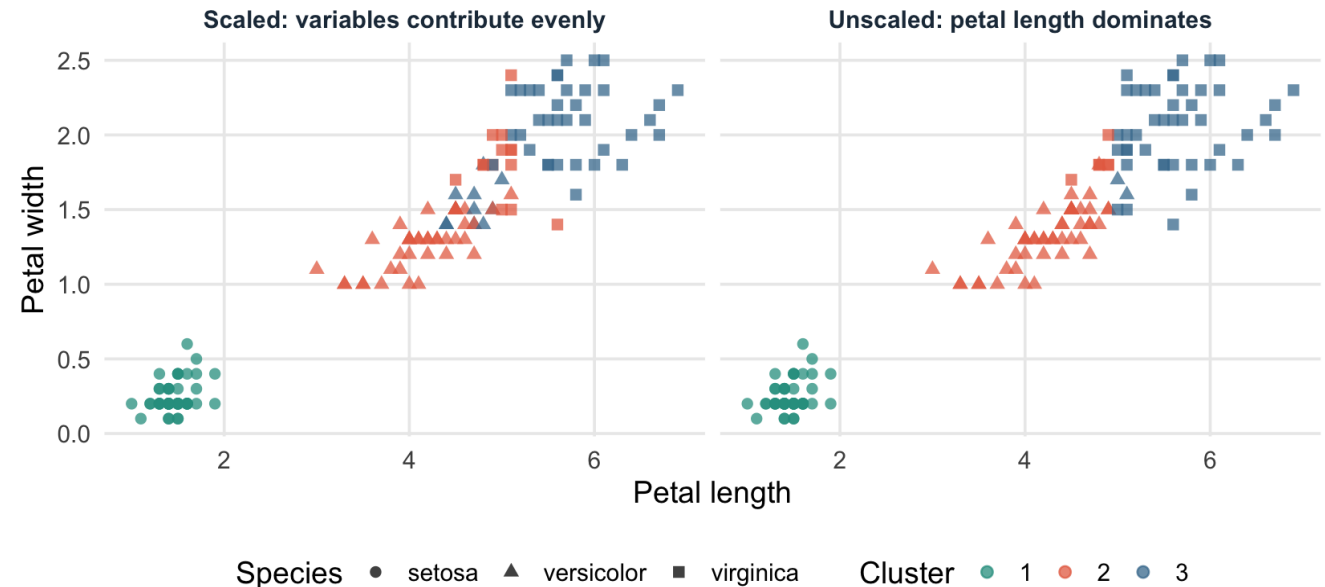
# Euclidean distance
euclidean_distances <- dist(iris_numeric, method = "euclidean")

# Manhattan distance
manhattan_distances <- dist(iris_numeric, method = "manhattan")
```

Scaling

Making sure features are on a comparable scale.

- Distance calculations are sensitive to feature magnitude.
- If one feature has much larger values than the others, it can dominate the distance calculation.
- Ex: if one column ranges from 0-1 and another ranges from 0-10,000 - Euclidean distance will reflect entirely the larger-scale column.
- Scaling helps each feature contribute more evenly to distance-based methods.
- Standardization is a common scaling technique - subtract the mean and divide by the standard deviation.



```
X <- iris[, 1:4]
X_scaled <- scale(X)
```

```
# Same shape as X, but each column is centered
# and standardized before distance/clustering.
summary(X_scaled)
```

Clustering

- *Find structure when you have no labels*

The question

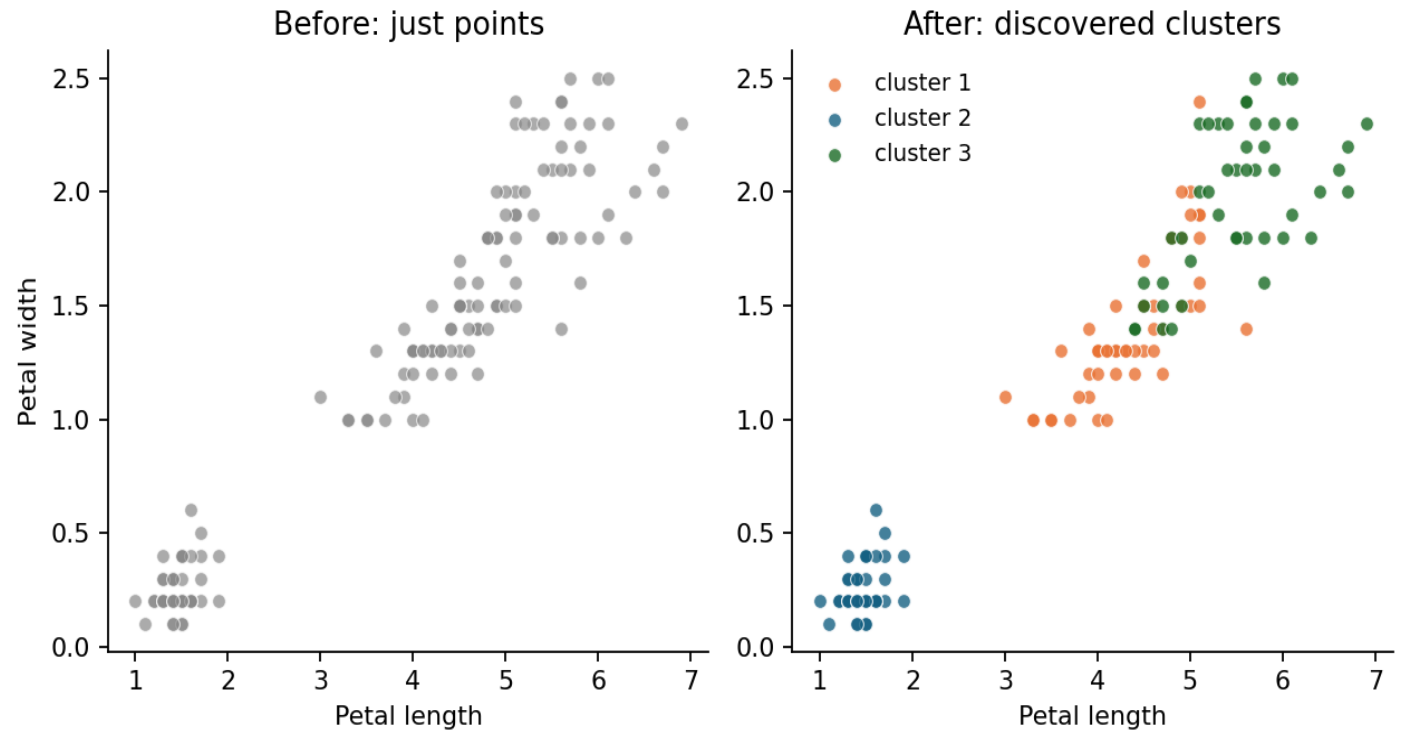
Given a set of points, which ones belong together?

The approach

Use distance to group close observations — no labels required (this is what makes it unsupervised).

The choice

Algorithms differ in how they define a group. We'll cover three families: centroid-based (K-means), tree-based (hierarchical), graph-based (Louvain/Leiden).



```
D <- dist(X_scaled) # pairwise distances
fit <- kmeans(X_scaled, centers = 3)
```

K-means

- *Partition into K compact, equal-variance groups*

How it works

- Pick K centers → assign points to nearest → update centers to the mean → repeat until stable.

When to use

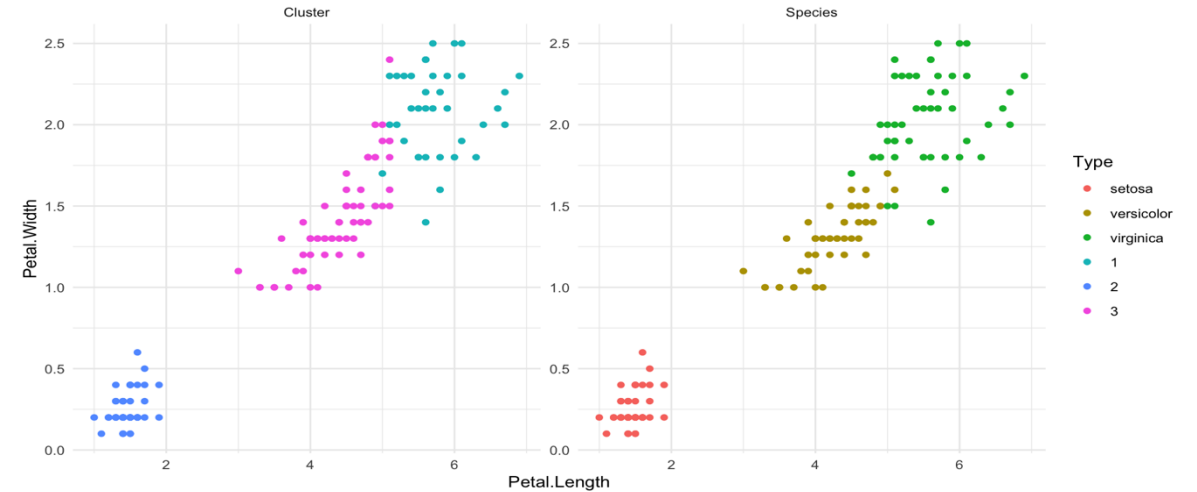
- Approximate K is known; clusters roughly spherical; speed at scale.

Strengths

- Fast ($O(n \cdot k \cdot i)$) and scales well
- Simple to explain
- Gives hard cluster labels

Watch out for

- Must choose K up front
- Sensitive to initialization (use $nstart \geq 25$)
- Assumes spherical equal-variance clusters
- Pulled by outliers



```
set.seed(123)
fit <- kmeans(X_scaled, centers = 3, nstart = 25)

fit$tot.withinss          # within-cluster SS
fit$cluster               # hard labels per row
table(fit$cluster, iris$Species)
```

Hierarchical Clustering

- *Build a tree of nested groupings — no K required*

How it works

- Start with every point as its own cluster.
- Repeatedly merge the two closest clusters; the full merge history is the dendrogram.

When to use

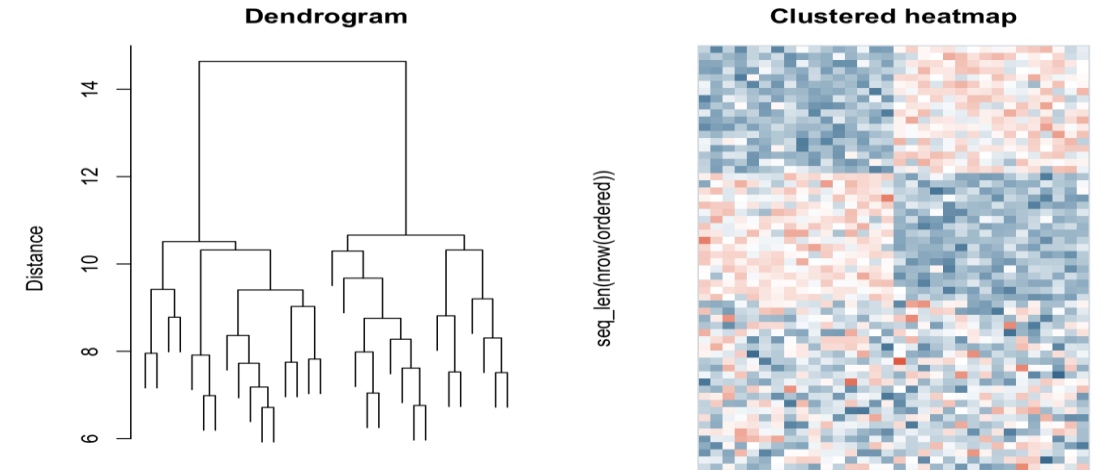
- You don't know K ; want nested structure; $n \leq \sim 10k$ (memory is $O(n^2)$).
- A dendrogram lets you see nested similarity rather than one fixed k .

Strengths

- No K up front
- Dendrogram is informative on its own
- Cut the tree at any height for different granularity

Watch out for

- Doesn't scale well ($10k >$)
- Linkage choice changes result



```
D <- dist(X_scaled, method = "euclidean")
hc <- hclust(D, method = "ward.D2")
plot(hc, labels = FALSE)
abline(h = 8, col = "red", lty = 2)

clusters <- cutree(hc, k = 3)
table(clusters, iris$Species)
```

Heatmaps with Hierarchical Clustering

- *See the matrix and the clustering at once*

How it works

- pheatmap() plots a numeric matrix and hierarchically clusters rows and/or columns.
- Similar profiles end up adjacent, so the matrix reads as a sorted picture.

When to use

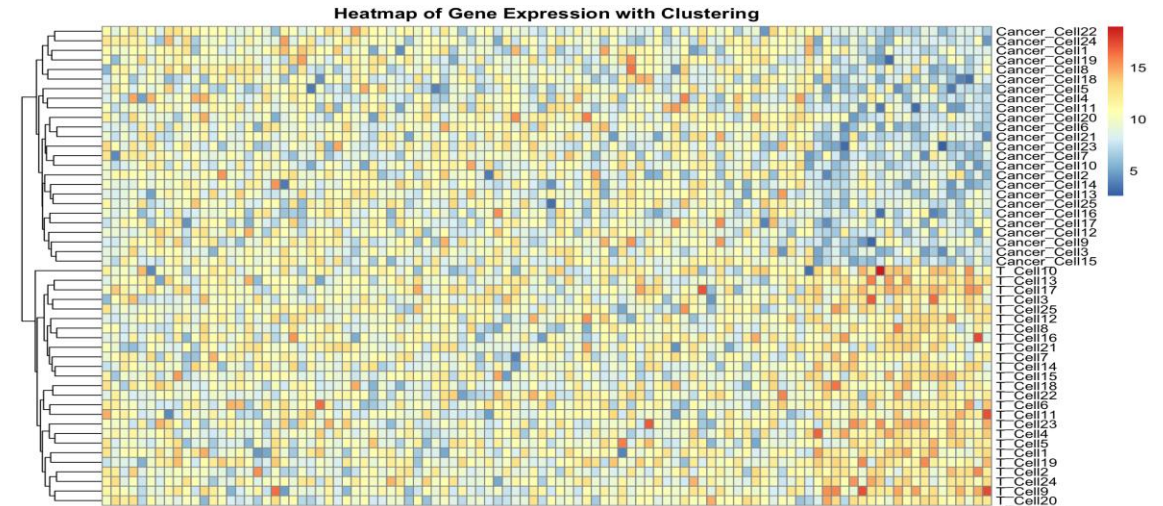
- Gene expression matrices; sample × feature views; finding co-regulated groups.

Strengths

- Clusters + raw values + metadata in one figure
- Row-scaling reveals patterns hidden by magnitude
- Excellent for publication figures

Watch out for

- Sensitive to scale and linkage method
- Hard to read past ~100 rows
- Color limits can mislead



```
library(pheatmap)

annot <- data.frame(Species = iris$Species,
                    row.names = rownames(X_scaled))

pheatmap(X_scaled,
          scale           = "row",
          clustering_method = "ward.D2",
          annotation_row   = annot,
          show_rownames    = FALSE)
```


Network-based Clustering

- *Cluster the graph instead of the points (Louvain / Leiden)*

How it works

- Build a k-NN graph from (usually reduced) data.
- Find dense communities by maximizing modularity (Louvain) or its refined cousin (Leiden).

When to use

- Large n (10^5 – 10^6); high-D data; single-cell RNA-seq; clusters of unequal size.

Strengths

- Scales to millions
- No K — granularity controlled by resolution
- Handles arbitrary cluster shapes
- Standard in single-cell

Watch out for

- Sensitive to k and resolution
- Two-step pipeline = two places to tune

```
library(FNN); library(igraph)

knn <- get.knn(X_scaled, k = 10)
edges <- cbind(rep(1:nrow(X_scaled), each = 10),
               as.vector(t(knn$nn.index)))
g <- graph_from_edgelist(edges, directed = FALSE)

clusters <- cluster_leiden(g, resolution_parameter =
1.0)$membership
```

Dimensionality Reduction

- *From many features to a few*

The problem

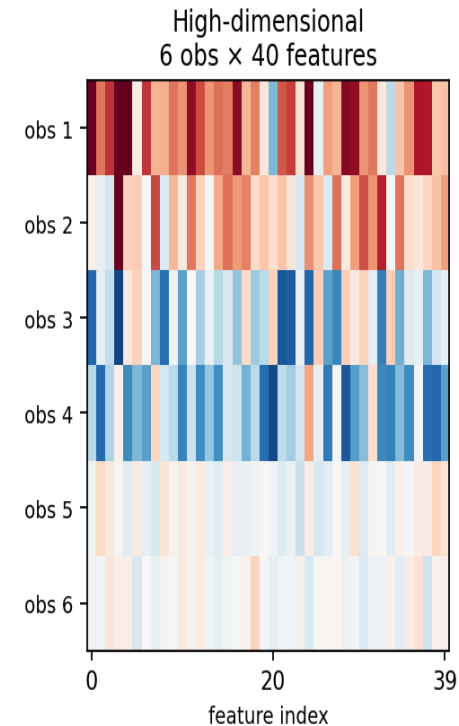
Real data has hundreds or thousands of features — too many to plot, too noisy to model directly.

The idea

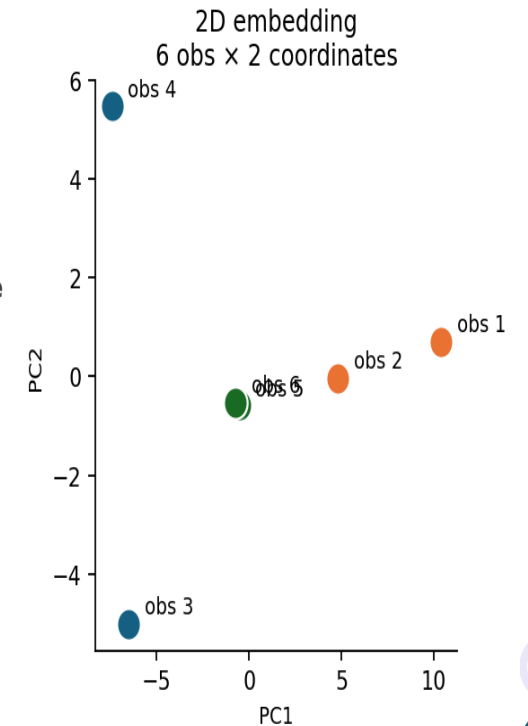
Project the data into a smaller number of coordinates while preserving the structure that matters.

Two families

Linear (PCA) preserves overall variance and gives interpretable axes. Non-linear (UMAP, t-SNE) preserves local neighborhoods and handles curved manifolds.



reduce



```
pca <- prcomp(X_scaled, scale. = TRUE) # linear (PCA)
um  <- umap(X_scaled)$layout           # non-linear (UMAP)
```

Principal Component Analysis (PCA)

- *Linear axes of maximum variance*

How it works

- Find orthogonal axes that capture the most variance, ranked by variance explained.
- PC1 is the direction of greatest spread; PC2 $\perp$ PC1 captures the next.

When to use

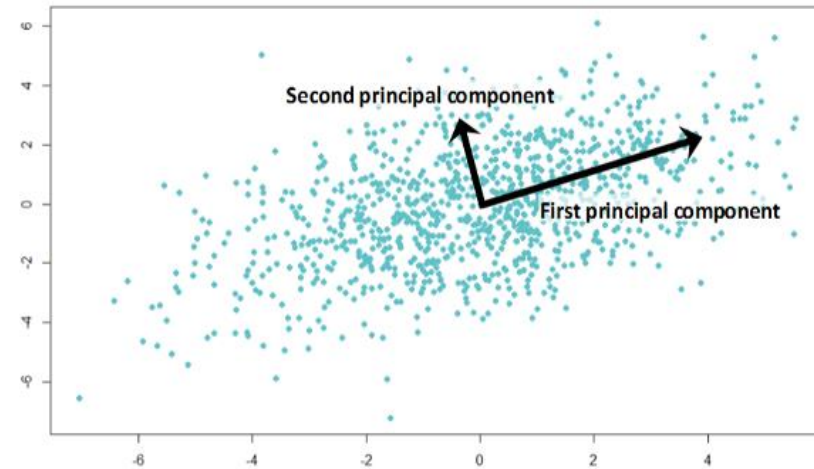
- Linear dim reduction; denoising before downstream methods; interpretable loadings.

Strengths

- Deterministic and fast
- No tuning
- Loadings show which features drive each PC
- Preserves global variance structure

Watch out for

- Scaling Input Values
- PCs can capture batch effects, etc.



```
pca <- prcomp(X_scaled, scale. = TRUE)

summary(pca)$importance[, 1:4]    # variance per
PC
plot(pca$x[, 1:2], col = iris$Species, pch = 19)
biplot(pca, cex = 0.6)            # PCs +
loadings
```

UMAP

- *Non-linear embedding that preserves local neighborhoods*

How it works

- Build a probabilistic nearest-neighbor graph in high-D space.
- Optimize a low-D layout (usually 2D) that preserves those neighborhoods.

When to use

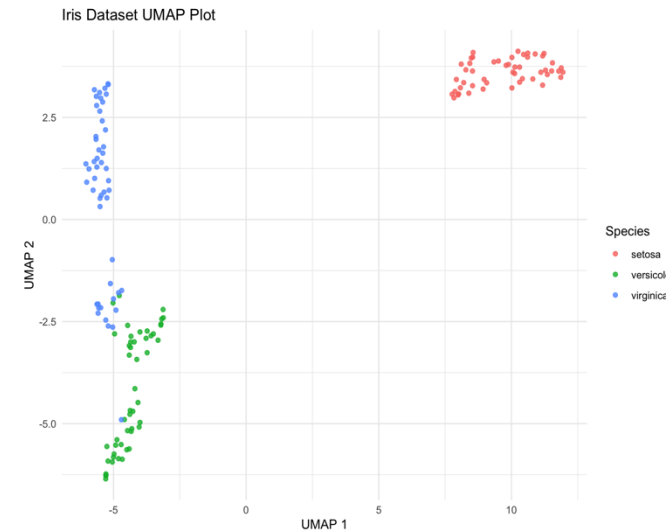
- Visualizing complex non-linear structure; single-cell embeddings; exploration before clustering.

Strengths

- Preserves local structure well
- Faster than t-SNE
- Retains more global structure than t-SNE

Watch out for

- Distances in the embedding are NOT meaningful
- Visual clusters can be artifacts of `n_neighbors` / `min_dist`
- Stochastic



```
library(umap); set.seed(42)

um <- umap(X_scaled,
           n_neighbors = 15,      # local-vs-global tradeoff
           min_dist     = 0.1)    # tightness of clusters

plot(um$layout, col = iris$Species, pch = 19,
     xlab = "UMAP1", ylab = "UMAP2")
```

Correlation & Regression

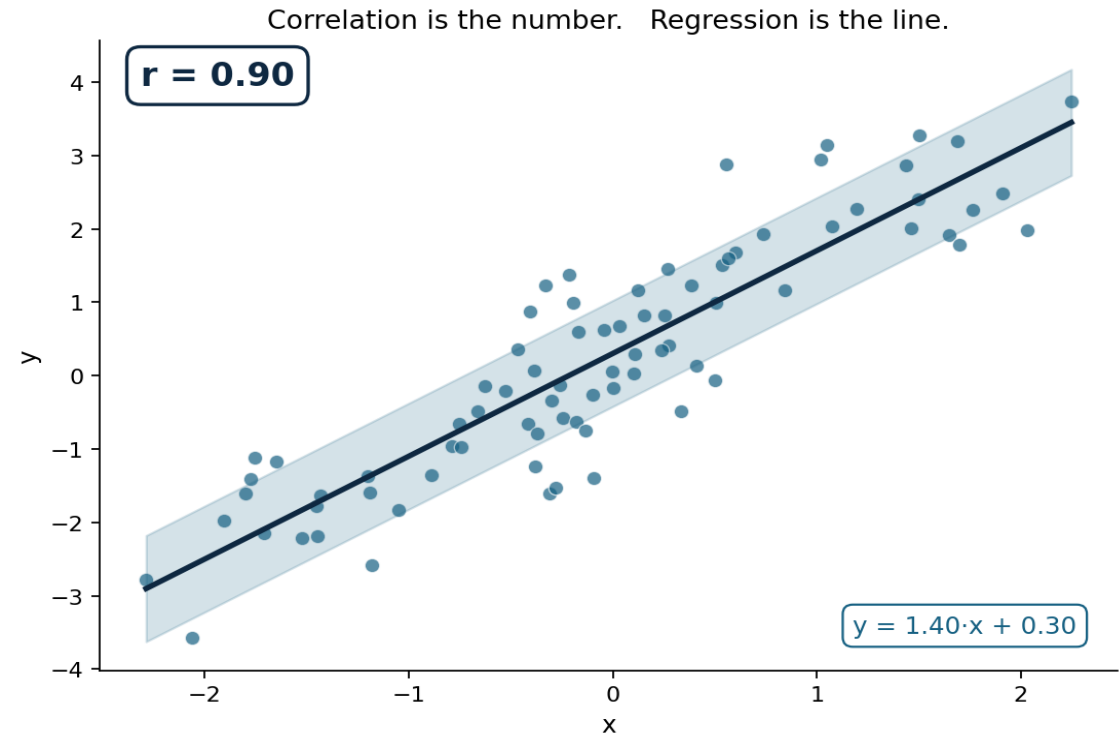
- *Measuring and modeling relationships between variables*

Correlation — do two variables move together?

A single number between -1 and +1 describing strength and direction.
Makes no assumption about cause.

Regression — can we predict one from the other?

Fits a line through the data. Gives an intercept, a slope, standard errors and p-values.



```
cor(x, y, method = "pearson")  # strength of relationship
lm(y ~ x, data = df)           # line of best fit
```

Pearson vs Spearman

- *Choosing the right correlation method*

How they differ

- Pearson measures linear relationship and assumes the variables are normally distributed.
- Spearman ranks both variables first, then correlates — captures any monotonic relationship.

When to use Pearson

- Continuous variables, roughly normal, relationship looks linear.

When to use Spearman

- Outliers or non-normal data; rank or ordinal data; relationship is monotonic but not necessarily linear.

In ggplot2

- `geom_smooth()` overlays a fitted model: `method = "lm"` for a line, `"loess"` for a curve.
- Add confidence bands with `se = TRUE` (default).

```
cor(x, y, method = "pearson")
cor(x, y, method = "spearman")

ggplot(df, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE)
```

Linear Regression (lm)

- *Predict y from x by fitting a line*

How it works

- Ordinary least squares — find coefficients that minimize the sum of squared residuals.
- Returns intercept, slopes, standard errors and p-values.

When to use

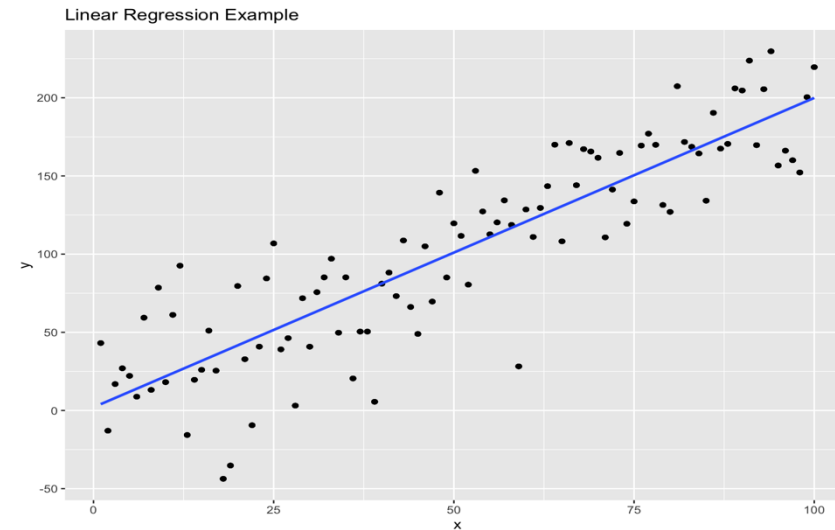
- Quantifying the effect of x on y ; prediction; hypothesis testing; baseline before complex models.

Strengths

- Closed-form and fast
- Interpretable coefficients with CIs and p-values
- Formula syntax extends to interactions

Watch out for

- Assumes linearity and homoscedasticity
- Outliers pull the line
- Correlation $\neq$ causation



```
model <- lm(Sepal.Length ~ Petal.Length + Species,
            data = iris)
summary(model)      # coefficients, R2, p-values
confint(model)      # confidence intervals
plot(model)          # 4 residual diagnostics
predict(model, newdata = new_df, interval = "confidence")
```

Conclusion

- Clustering
 - Grouping similar data points together into meaningful clusters.
 - Similarity is measured by a distance function.
- Dimensionality reduction
 - Reducing high-dimensional data into a compact set of explanatory features.
 - Useful for visualization and noise removal.
- Correlation
 - Measuring the strength and direction of relationships and plotting the relationships using `geom_smooth`.

